

Contributions individuelles

MediPlan — projet intégrateur 030747 · Collège La Cité · printemps 2026

Ce document sert deux usages : établir qui a fait quoi, et servir de fiche de révision avant la présentation. **Chacun doit pouvoir répondre à une question sur sa propre colonne** sans se retourner vers les autres.

Tout ce qui suit est vérifiable dans le dépôt (git log, pull requests) et dans Jira. Les attributions ont été retracées commit par commit, pas de mémoire.

Vue d'ensemble

	Souleymane DIALLO	Zakaria Lahouiri	Larbi Saib
Rôle tenu	Pilotage, socle technique, sécurité, mise en ligne, espace patient	Disponibilités, flux du jour, notifications	Rendez-vous, patient, statistiques
Conception	Cahier des charges, 7 cas d'utilisation	Diagramme de classes	Diagrammes de séquence
Commits sur main	72	8	4
Épiques Jira	E1, E2, E4, E7	E3, E5, E6	—

Le volume de commits est très inégal, et nous l'assumons. Souleymane a porté le socle technique et l'intégration, ce qui produit mécaniquement beaucoup de commits. Zakaria et Larbi ont livré des fonctionnalités complètes en peu de commits. Le nombre de commits mesure une façon de travailler, pas une contribution.

Souleymane DIALLO

Ce dont j'étais responsable

Le socle technique, la sécurité, et tout ce qui relie les morceaux entre eux.

Ce que j'ai réalisé

Conception — cahier des charges, les 7 cas d'utilisation, l'ERD, la mise en place du dépôt et la remise du dossier *(MEDIPLAN-8 à 14, 31 à 33)*.

Socle technique — monorepo Turborepo + pnpm, conteneurisation Docker Compose, CI GitHub Actions *(MEDIPLAN-28, 29, 30 — PR #1)*.

Authentification et sécurité — inscription, connexion JWT, hachage bcrypt, verrouillage de compte après échecs répétés, réinitialisation du mot de passe, contrôle d'accès par rôle à 4 rôles avec périmètre de clinique *(MEDIPLAN-15, 16, 17 — PR #2, #6, #7)*.

Frontend — écrans d'authentification, coquille applicative, design system (Angular Material 3 + Tailwind), garde de rôle et masquage de la navigation, refonte UI complète, mode sombre *(MEDIPLAN-41 à 48, 51 — PR #3, #4, #8, #9, #10, #11)*.

Index anti-double-réservation partiel — j'ai repris la contrainte d'unicité posée par Larbi et l'ai transformée en **index unique partiel**, excluant les rendez-vous annulés. C'est ce qui rend l'annulation possible : sans cela, un créneau annulé restait bloqué à jamais *(migration 1781674897615, intégration du 8 juillet)*.

Annulation d'un rendez-vous avec motif obligatoire *(MEDIPLAN-22 — PR #15)*.

Réservation par le patient en libre-service — le second canal : annuaire public des cliniques, choix de sa clinique à l'inscription, consultation des créneaux réellement libres, réservation, « Mes rendez-vous ». La réservation emprunte **la même transaction, le même verrou et le même index** que celle de la réception : la garantie anti-double-réservation ne dépend pas du canal *(MEDIPLAN-21 — PR #25, #26)*.

Mise en ligne — infrastructure Azure entièrement décrite en Bicep, base PostgreSQL infogérée, publication des images, coût maintenu à environ 0 \$/mois *(MEDIPLAN-40 — PR #16, #17, #18)*.

Intégration finale — réintégration des branches de Zakaria et Larbi, remise en cohérence de Jira avec le dépôt *(PR #19, #20, #21)*.

Ma principale difficulté

Un bogue qui n'existait qu'en production. Une fois l'application en ligne, tous les appels à l'API renvoyaient 404 — alors que tout fonctionnait parfaitement en local.

Comment je l'ai résolue

J'ai d'abord cherché dans le code, puis dans les routes, puis dans la configuration. Sans résultat, parce que le problème n'y était pas.

J'ai changé de méthode : au lieu de chercher **ce qui était cassé**, j'ai cherché **ce qui différait entre les deux environnements**. La réponse était dans le routage. Notre proxy transmettait l'en-tête Host demandé par le navigateur ; **Azure Container Apps route selon cet en-tête**, alors que Docker Compose, en local, ne route pas du tout de cette façon.

Ce que j'en retiens

Un environnement de développement qui fonctionne ne prouve rien sur la production. Les deux ne diffèrent pas seulement par leurs données — ils diffèrent par leur façon d'acheminer une requête. J'ai revu le même schéma deux fois depuis : les fins de ligne Windows qui empêchaient un conteneur

de démarrer, et les horaires de la clinique décalés de quatre heures parce que le conteneur tourne en UTC.

Questions auxquelles je dois savoir répondre

- Pourquoi le backend n'a-t-il pas d'adresse publique ? *(ingress interne : seul le frontend l'appelle, donc aucune requête inter-origines, donc aucun CORS)*
- Comment les secrets sont-ils gérés ? *(paramètres Bicep @secure() puis secrets natifs Container Apps ; rien dans le dépôt)*
- Pourquoi des migrations plutôt que la synchronisation automatique ?
(reproductibilité : n'importe qui reconstruit exactement le même schéma)
- Pourquoi Azure et pas Railway comme prévu au départ ? *(crédit étudiant non renouvelable ; scale-to-zero, coût ~0 \$/mois)*
- Un patient peut-il réserver au nom de quelqu'un d'autre ? *(non : le corps de la requête n'expose aucun identifiant de patient, l'identité vient du jeton)*
- Pourquoi un patient ne peut-il pas annuler lui-même ? *(l'annulation suppose une règle de délai minimum, UC-07, non implémentée ; nous préférons ne pas livrer l'une sans l'autre)*

Zakaria Lahouiri

Ce dont j'étais responsable

Le temps du médecin : ses disponibilités, sa journée, et ce qui l'informe.

Ce que j'ai réalisé

Conception — le diagramme de classes *(commit 42f426d)*.

Disponibilités des médecins — plages datées, génération automatique des créneaux reservables à partir d'une plage et d'une durée, plages de congé *(MEDIPLAN-20 — commit 11b98ad)*.

Flux clinique du jour — la vue partagée entre la réception et le médecin, avec les transitions de statut du rendez-vous *(MEDIPLAN-23 — commit 6b40bcb)*.

Notifications internes — module complet : entité, migration, endpoints, et émission automatique sur les trois événements du cycle de vie du rendez-vous (réservation, changement de statut, annulation). Côté interface, la cloche avec son compteur de non-lues *(MEDIPLAN-25 — PR #21)*.

Export CSV des rendez-vous sur une période, borné à la clinique de l'appelant *(MEDIPLAN-27 — PR #20)*.

Configuration de clinique *(MEDIPLAN-18)* et **décalage en bloc des rendez-vous** *(MEDIPLAN-24)* : codés et testés sur des branches, **non intégrés** — voir la difficulté ci-dessous.

Ma principale difficulté

Mes branches avaient dérivé. J'ai travaillé longtemps sans fusionner. Quand il a fallu intégrer, certaines de mes branches accusaient **jusqu'à 56 commits de retard** sur main — dont une refonte complète de l'interface. Le code était bon, mais il ne s'appliquait plus sur rien de reconnaissable.

Comment je l'ai résolue

Branche par branche, rebasage sur main, puis résolution des conflits **un par un**. La règle que nous nous sommes donnée : **conserver les deux côtés**. Ces conflits n'étaient pas des désaccords, c'étaient deux ajouts au même endroit.

Un cas concret : sur le service des rendez-vous, git avait fusionné ma méthode d'export et la méthode d'historique de main en une seule méthode incohérente. Il a fallu les rétablir séparément.

Deux de mes branches n'ont pas été reprises — configuration de clinique et décalage en bloc. À trois jours de la présentation, les rejouer par-dessus l'interface refondue faisait courir un risque de régression sur le cœur du produit. **Les tickets sont repassés « À faire » dans Jira**, avec la raison écrite.

Ce que j'en retiens

Un ticket n'est pas terminé quand le code est écrit. Il est terminé quand il est fusionné. Une branche qui vit trois semaines coûte plus cher à intégrer qu'elle n'a coûté à écrire.

Questions auxquelles je dois savoir répondre

- Comment les créneaux sont-ils générés ? *(à partir d'une plage datée et d'une durée ; le nombre est annoncé avant validation)*
- Qui reçoit une notification, et quand ? *(les personnes concernées par le rendez-vous, à la réservation, au changement de statut et à l'annulation)*
- Où sont déclarées votre entité et votre migration ? *(explicitement dans `data-source-options.ts`, jamais par glob — convention du projet)*
- Pourquoi le décalage en bloc n'est-il pas là ? *(codé, non intégré, décision assumée à trois jours de l'échéance)*

Larbi Saib

Ce dont j'étais responsable

Le rendez-vous lui-même : le patient, la réservation, et la mesure de l'activité.

Ce que j'ai réalisé

Conception — les diagrammes de séquence `*(commit 2b4ac71)*`.

Modèle du « patient léger » — un patient créé au comptoir par la réception, sans compte ni mot de passe, et volontairement non connectable. C'est la traduction dans le modèle de données du fait qu'un patient de clinique appelle, il ne s'inscrit pas `*(MEDIPLAN-35 — commit 03cafbcb)*`.

Prise de rendez-vous par la réception — le flux complet : entités Appointment et AppointmentSlot, migration, service, contrôleur, validation `*(MEDIPLAN-36, 49 — commit 3f67d8d)*`.

Garantie anti-double-réservation — la contrainte d'unicité en base sur le créneau, posée dès la première version `*(migration 1781674897614)*`.

Tableau de bord et statistiques — module backend d'agrégation et écran d'administration : volume de rendez-vous, taux d'absence, taux d'occupation, détail par médecin. C'est ce qui a remplacé les compteurs factices du tableau de bord par des chiffres réels `*(MEDIPLAN-26 — PR #19)*`.

Ma principale difficulté

Garantir qu'un créneau ne soit jamais réservé deux fois. Ma première approche était celle qui vient naturellement : vérifier que le créneau est libre, puis écrire le rendez-vous.

Comment je l'ai résolue

En comprenant pourquoi cette approche ne peut pas marcher. **Entre la vérification et l'écriture, une autre requête peut passer.** Les deux voient le créneau libre, les deux réservent. Le défaut n'apparaît que sous charge — donc jamais pendant les tests manuels, et toujours le jour où deux réceptionnistes travaillent en même temps.

J'ai donc sorti la garantie du code applicatif pour la poser **dans la base**, sous forme de contrainte d'unicité sur le créneau. Ce n'est plus notre code qui arbitre la course, c'est PostgreSQL — et lui ne se trompe pas.

Souleymane a ensuite fait évoluer cette contrainte en **index unique partiel**, excluant les rendez-vous annulés, ce qui a permis d'implémenter l'annulation avec libération du créneau.

Ce que j'en retiens

Quand une garantie dépend de la discipline du développeur, elle finira par tomber. Une garantie posée dans la base tient même si une future fonctionnalité oublie de vérifier.

Questions auxquelles je dois savoir répondre

- Que se passe-t-il si deux personnes réservent le même créneau en même temps ?

(la base refuse la seconde écriture ; vérifié sur l'application déployée : une requête en 201, l'autre en 409)

- Pourquoi le patient n'a-t-il pas de compte ? *(patient léger : il appelle la clinique, il ne s'inscrit pas en ligne)*
- Que mesurent vos statistiques, et pourquoi celles-là ? *(taux d'absence = temps médecin payé et non facturé ; taux d'occupation = faut-il ouvrir plus de plages)*
- Les statistiques sont-elles bornées par clinique ? *(oui, comme toutes les requêtes, par le rôle de l'appelant)*

Travail commun

Trois choses n'appartiennent à personne en particulier :

- **la décision de découper par tranche verticale** — chacun mène sa fonctionnalité de la migration jusqu'à l'écran, plutôt qu'un « un front / un back » ;
- **la définition de « terminé »**, revue en cours de projet : terminé = fusionné dans main, CI verte ;
- **la réflexion UX/UI** et la préparation de la présentation.